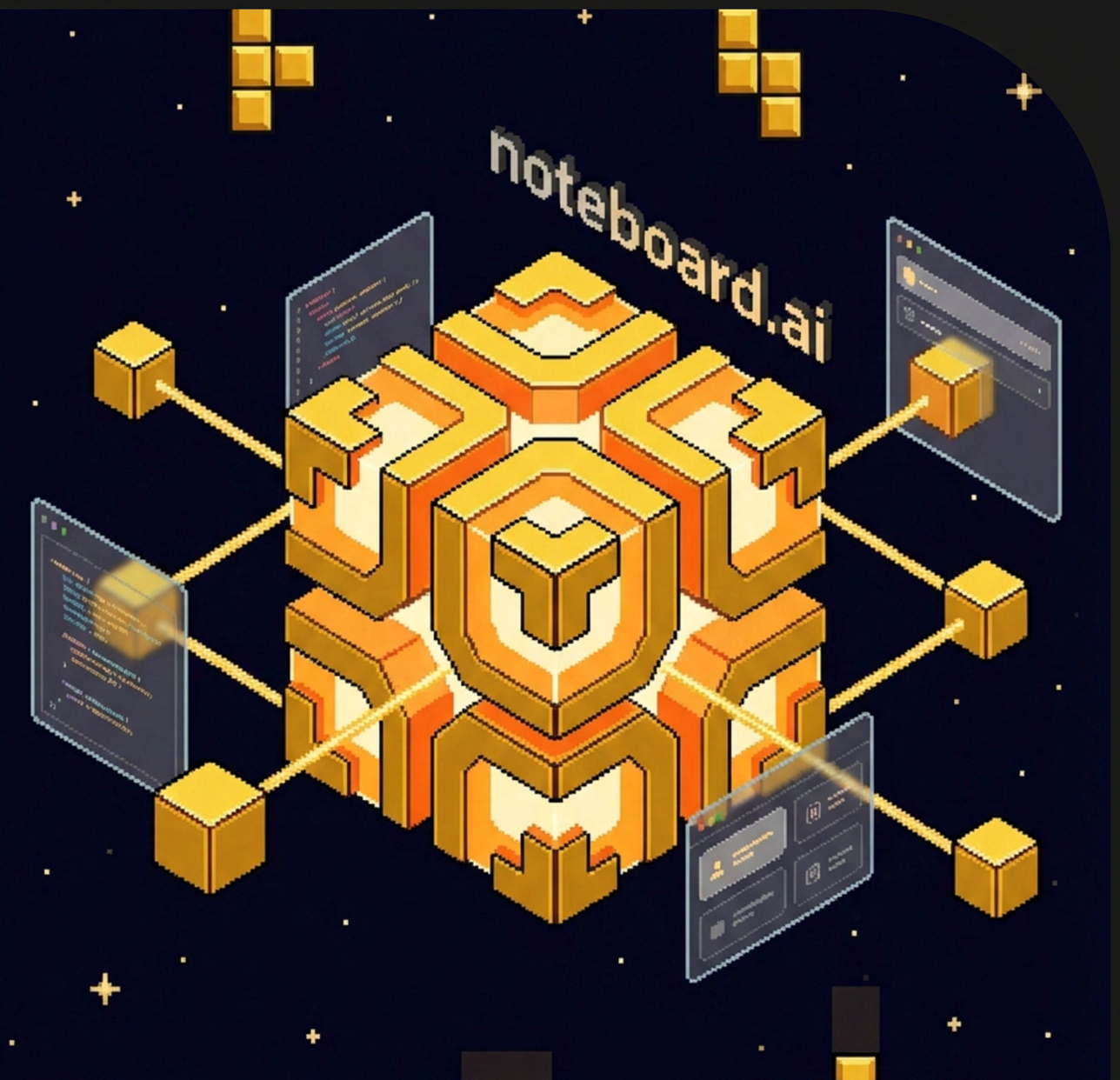


NOTEBOARD.AI

ABOUT ME

SARRTHAK TRIPATHI
MS Data Science
sarrtrip@iu.edu

“Experiencing the Tetris Effect”

“Software” Of The AI Era

“Software is eating the world.”

Marc Andreeson
General Partner Andreeson Horowitz
August 20th, 2011

“Software is eating the world, but AI is going to eat software.”

Jensen Huang
CEO NVIDIA Corp.
May 12th, 2017

The Problem: The Broken SDLC

The modern SDLC is an assembly line communicating using broken Telephones

The Fragmentation: Software creation today is sliced into silos—PMs define, Designers visualize, and Developers execute. Every handoff creates Context Loss. By the time a ticket reaches the developer, the "Why" is lost, leaving only the "What."

The Latency Trap: We measure success by "Velocity" (how fast we move tickets), not "Value" (how much capability we ship).

The Cognitive Load: A single developer is forced to context-switch between infrastructure, frontend pixels, and backend logic, turning creativity into administrative exhaustion.

The Verdict: We are using an industrial-era assembly line process for a creative digital discipline. The friction isn't in the code; it's in the process.

What Market Does?: The World

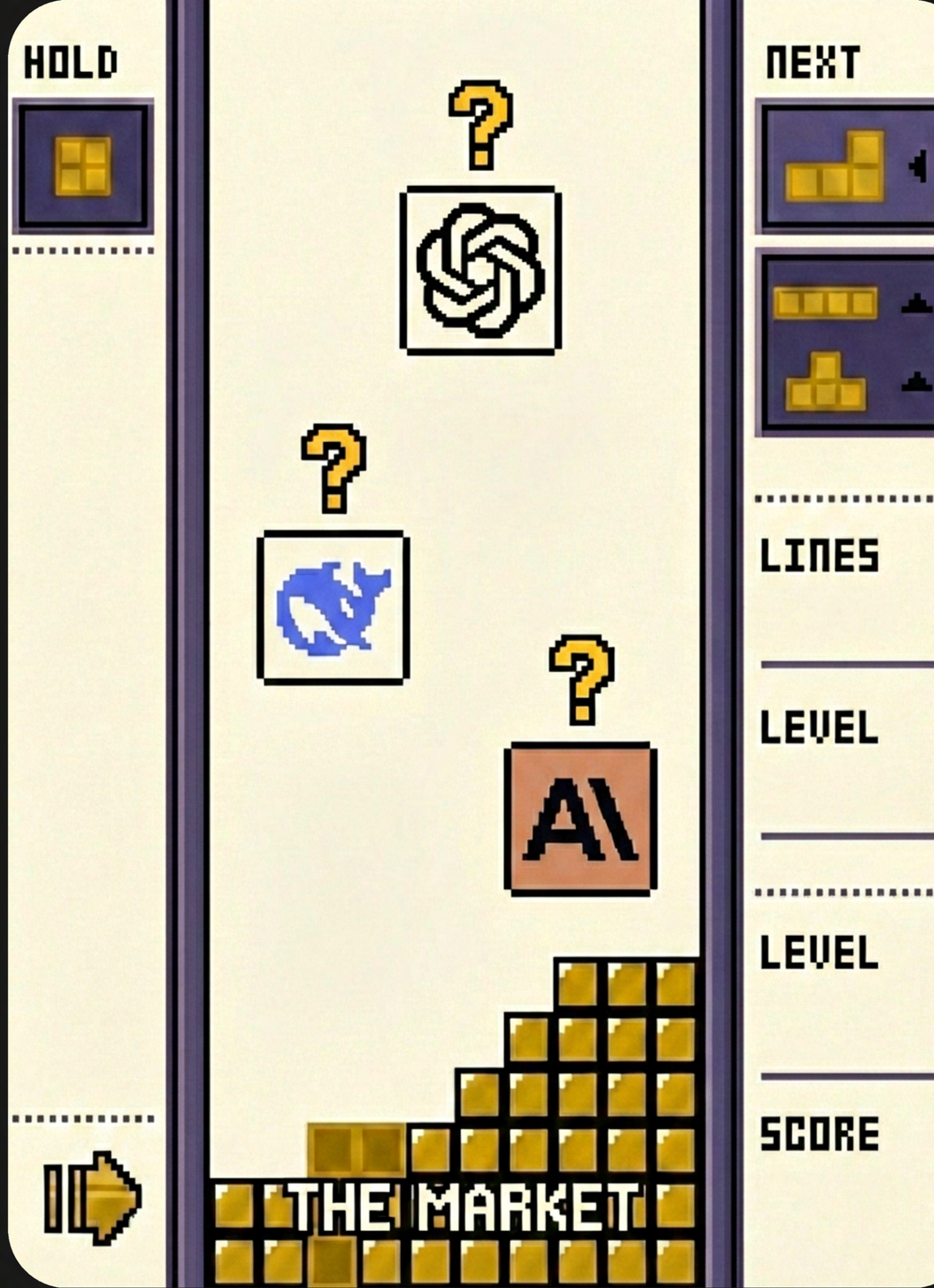
What the Industry is doing (The Trap):

- Current AI tools (like autonomous coders) are trying to replace the engineer)
- They optimize for Code Generation —writing syntax faster and Generating Code. **The AI knows the Tech but not the Business.**
- The Flaw: This assumes the requirements are perfect. But code without empathy creates products that work technically but fail the user. AI understands machines.

What Market Does?: The World

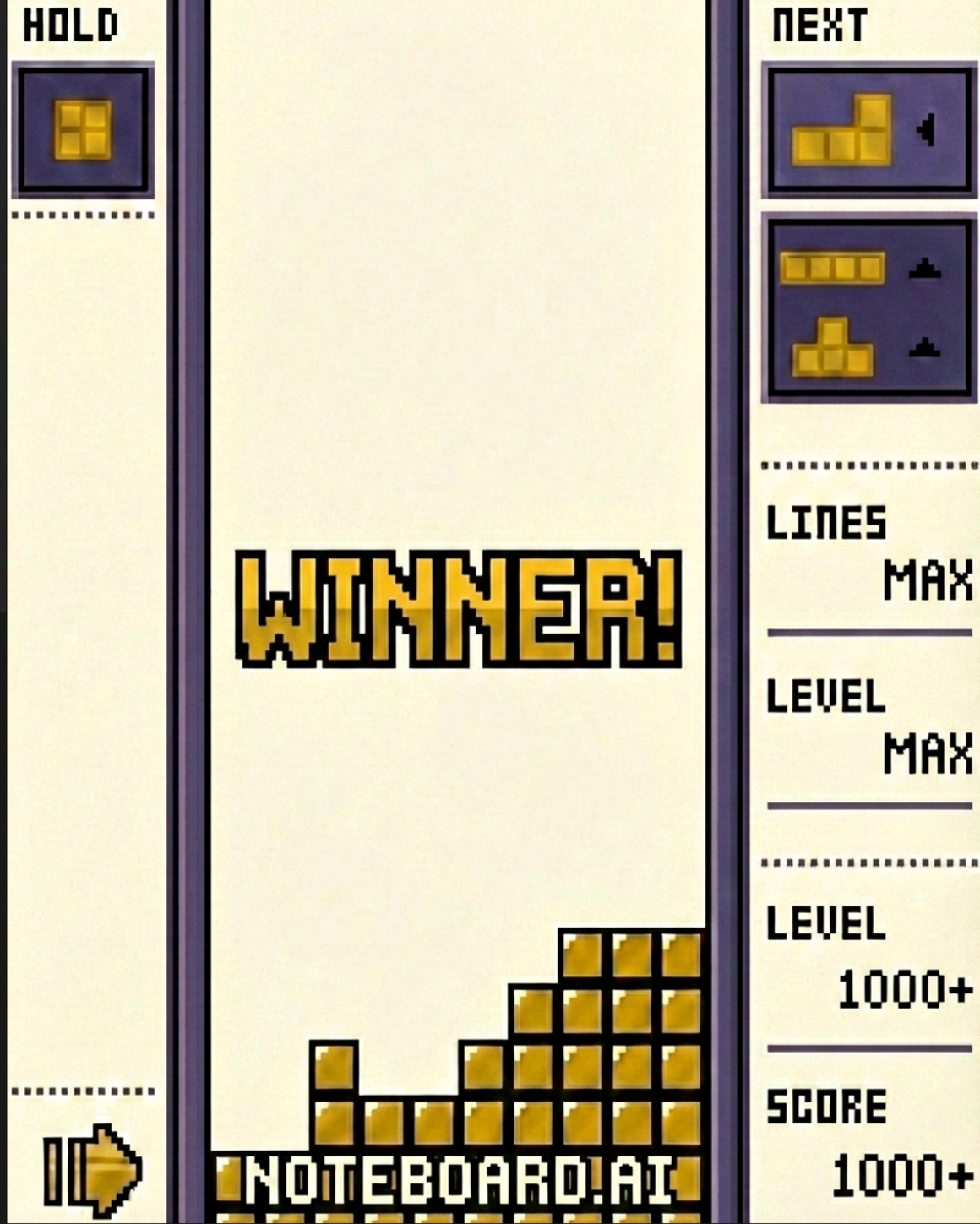
AI understands machines but not Humans

- *The Current Tools are Good at executing ideas, but Hallucinate when making key Architecture Decisions.*
- *The Blind Spot: Competitors are trying to replace the developer completely with AI. This is a mistake. Code without human empathy creates products that function technically but fail users.*



What We Do: An Idea

AI-orchestrated SDLC platform transforming developers into Directors, enabling "One Person, One Product" through capability-driven automation.



What We Do?: A Deep Dive

Noteboard.ai is the first Context-Aware SDLC Platform. Unlike existing tools that only automate code generation (solving for syntax), we automate the software lifecycle (solving for value).

We reengineer the development process into three AI-augmented views:

Huddle: Voice-to-Intent translation that builds a real-time Knowledge Graph of business capabilities using Neo4j.

Design: Generative Architecture that creates executable HLD/LLD diagrams using Vision Models.

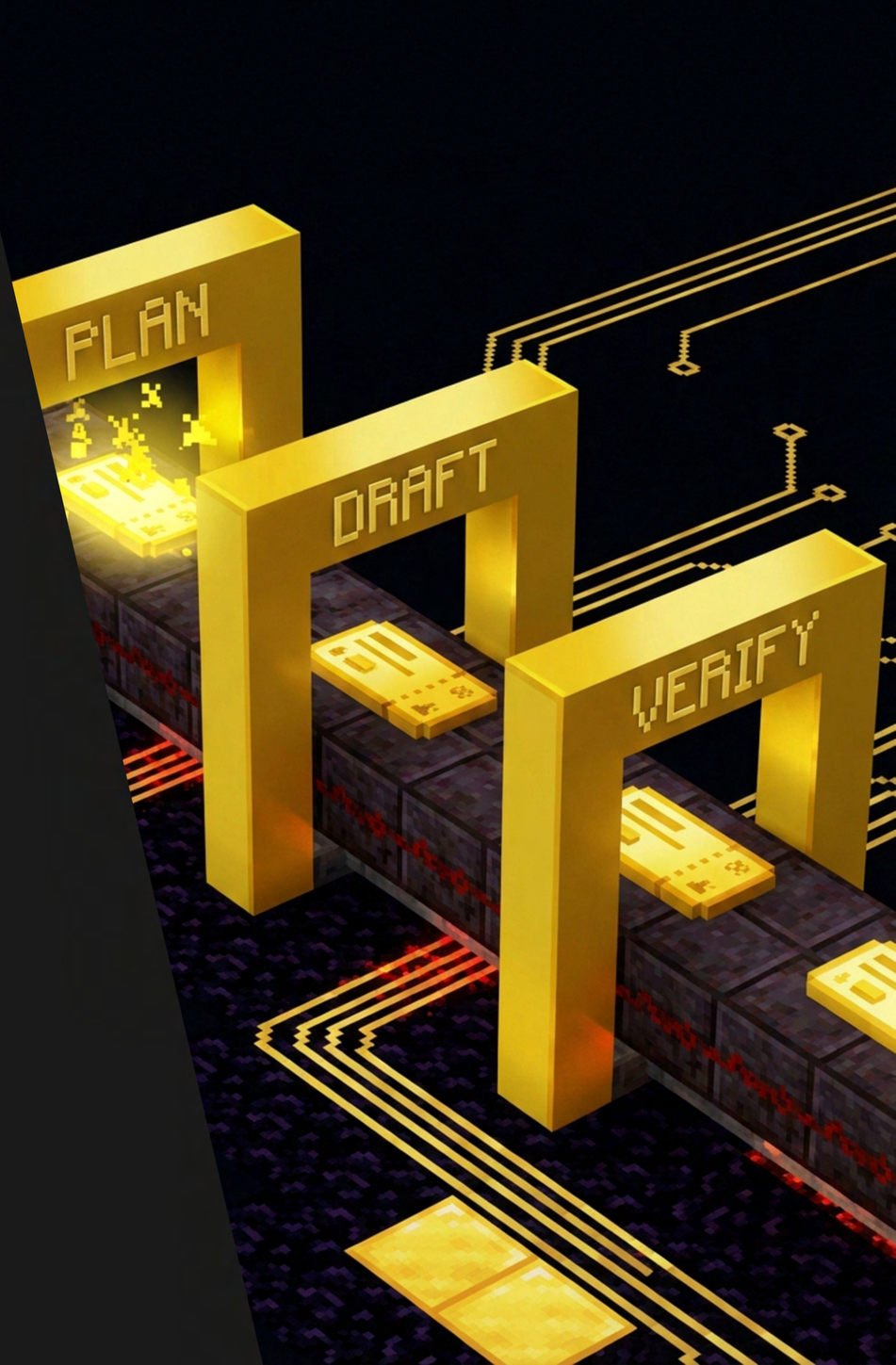
Dev: An agentic "Mission Control" where the developer **supervises autonomous agents via a Checkpoint System rather than writing every line of code.**

By shifting the unit of work from "Lines of Code" to "Business Capabilities," we allow a single developer to own the full stack without cognitive burnout.

The Solution: The "Intelligent Kitchen" Workflow.

What We Do?:A Deep Dive

- We address the fragmentation problem by collapsing the SDLC into a single, fluid interface:
  - **Preserving Intent:** Instead of JIRA tickets, we use a Knowledge Graph (The Pantry). When a developer defines a feature in the "Huddle," it links to every other dependency in the system. The AI knows that "Checkout" relies on "Auth" because the graph says so.
- **Collapsing Latency:** We use AI Agents (The Stove) to execute the "boring" parts—boilerplate, testing, and infrastructure—instantly.
- **The Director Model:** The developer acts as a Supervisor. They approve the "Plan," review the "Design," and verify the "Build" via checkpoints. This keeps the Human in the Loop for high-level reasoning while offloading execution to the machine.



What We Do?: Under The Hood

A Detailed Discussion in the Solution

Our architecture is a Modular Monolith designed for high-throughput agentic workflows:

- The Semantic Layer: We use Neo4j (Graph DB) to store "Capabilities" and their dependencies, and Weaviate (Vector DB) for RAG to retrieve historical design patterns. This gives our Agents "Long-term Memory."
- The Agentic Core: We utilize LangGraph to build stateful agents. Unlike simple chatbots, our agents operate on a State Machine:

(Plan → Draft → Verify →Deploy)

- Allowing for human intervention at specific gates.
- Multimodal Inputs: We use OpenAI Whisper v3 for high-fidelity voice diarization in the "Huddle" view and GPT-4o (Vision) to analyze and generate architecture diagrams in the "Design" view.

- Infrastructure: The system runs on Dockerized Microservices (FastAPI for logic, Next.js for UI, Celery/Redis for Async Tasks) deployed on AWS EC2, ensuring a scalable, production-ready environment.

What We Do?: LDL Diagram

Until Now & Future Work

Current Stage of Developlment

Current Status: Phase 1 (Iron Skeleton) Complete.

- Infrastructure: Docker orchestration, CI/CD pipelines via GitHub Actions, and AWS provisioning are live.
- Huddle View (Alpha): The Voice-to-Text pipeline is functional. The "Strategist" Agent successfully converts unstructured transcripts into JSON Capabilities and visualizes them in the Knowledge Graph (Pantry).

Team Roles:

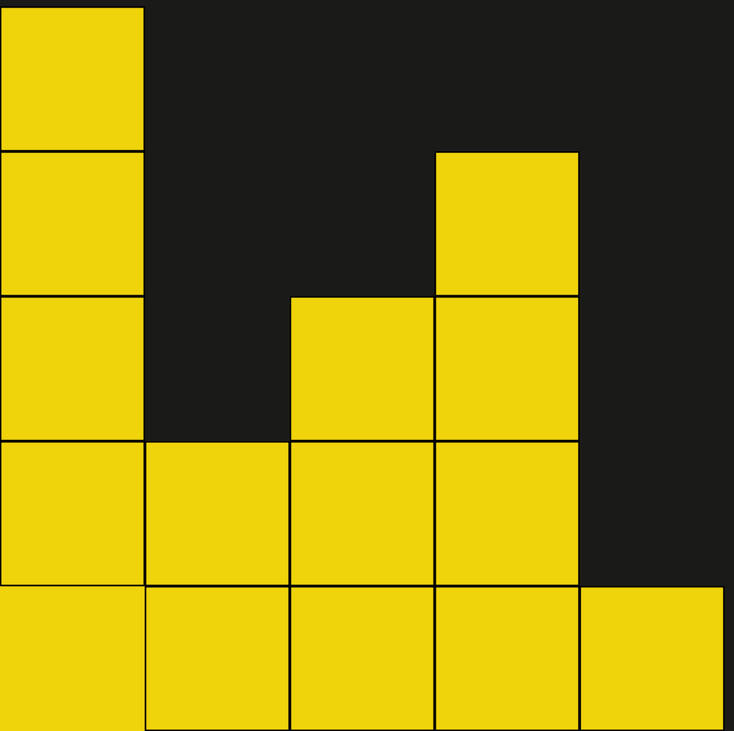
- Sarrthak Tripathi:
  - Engineering Architect – Responsible for the LangGraph state machines, FastAPI architecture, and Neo4j integration.
  - Product Architect - Responsible for the "Intelligent Kitchen" UX, Next.js implementation, and enforcing the "Business Value" metric.

Our Timeline & Future Work

Using our Runway

The competition funds will be strictly allocated to accelerate technical velocity:

- ***40% - AI Compute & API Costs:*** *High-volume usage of Claude 3.5 Sonnet (for reasoning) and GPT-4o (for vision) during the intense development and testing of our agents.*
- ***30% - Infrastructure:*** *AWS costs for hosting the Neo4j AuraDB instance and EC2 GPU instances for local model inference (DeepSeek-Coder) to reduce latency.*
- ***20% - Synthetic Data Generation:*** *Creating a massive dataset of "Golden Architectures" and "Failure Logs" to fine-tune our agents, solving the "Cold Start" problem.*
- ***10% - Developer Tooling:*** *Licenses for monitoring tools (BetterStack) to trace complex agentic failures during the prototype phase.*



Using Our Runway

- **Miscellaneous - User Research & Controlled Testing:** We will allocate funds to conduct deep-dive surveys with developers to identify specific SDLC bottlenecks. This includes running controlled testing of our alpha product to uncover friction points and validate our "1 Person, 1 Product" thesis.